Greedy Templates

Two interval templates plus non-interval greedy patterns.

Quick Reference Table

#	Name	Description	Intuition	Time	Space	Key Implementation
435	Non-overlapping Intervals	Remove the minimum number of intervals to make the rest non-overlapping.	Always keep the interval that finishes earliest — it leaves the most room for the rest, so greedily picking by end maximizes how many you keep.	O(n log n)	O(1)	Standard
452	Minimum Number of Arrows to Burst Balloons	Balloons span `[start, end]`. An arrow shot at `x` pops all balloons with `start ≤ x ≤ end`. Minimum arrows to pop all balloons.	Shoot the arrow at the earliest end point to pop every balloon overlapping it; only start a new arrow when a balloon begins past the current arrow.	O(n log n)	O(1)	Standard
646	Maximum Length of Pair Chain	Given pairs `[a, b]`, form a chain where `b < c` for each consecutive pair `[a,b]` → `[c,d]`. Maximum chain length.	Picking the pair that ends earliest each time leaves the most slack for later links, maximizing the chain.	O(n log n)	O(1)	Standard
56	Merge Intervals	Merge all overlapping intervals. Return the minimum set of non-overlapping intervals.	sort to make overlaps adjacent, then sweep once — each interval only ever compares against the most recently kept one.	O(n log n)	O(n)	Standard
57	Insert Interval	Insert a new interval into a sorted list of non-overlapping intervals; merge if necessary. The input is already sorted — no sort needed.	Walk the sorted list in three phases — copy everything that ends before the new interval, absorb everything that overlaps it, then copy the rest.	O(n)	O(n)	Standard
253	Meeting Rooms II	Minimum number of meeting rooms required to host all meetings.	USE WHEN counting simultaneous/overlapping resources — "minimum rooms", "max concurrent". The heap size at any moment is exactly how many meetings are running at once.	O(n log n)	O(n)	Standard
55	Jump Game	Each element is the max jump from that position. Can you reach the last index?	Sweep left to right tracking the farthest index reachable; if you ever stand past that frontier you're stuck.	O(n)	O(1)	Standard
45	Jump Game II	Minimum jumps to reach the last index.	Treat each jump as a BFS layer — extend through the whole current reach before incrementing the jump count.	O(n)	O(1)	Standard
763	Partition Labels	Partition string into as many parts as possible so each letter appears in at most one part. Return the size of each part.	A partition can't close until you pass the last occurrence of every letter seen inside it, so extend `end` to that frontier and cut when you reach it.	O(n)	O(1)	Standard
406	Queue Reconstruction by Height	People given as `[h, k]` (height h, k taller/equal people in front). Reconstruct the queue.	Place tallest first so that inserting a person at index `k` is always correct — only taller/equal people are already placed, and shorter insertions	$O(n^2)$	O(n)	Standard

#	Name	Description	Intuition	Time	Space	Key Implementation
			later don't disturb their counts.			
252	Meeting Rooms	Given an array of meeting intervals, determine if a person could attend all meetings (no two meetings overlap).	After sorting by start, any overlap must be between adjacent intervals, so one linear scan comparing neighbors is enough.	O(n log n)	O(1)	start < previous end = overlap
134	Gas Station	There are n gas stations on a circular route. `gas[i]` is the gas at station i; `cost[i]` is the gas needed to travel from i to i+1. Return the starting station index if you can complete the circuit, or -1 if not.	If the tank ever drops below 0, no station between the old start and here can work, so jump the start past the failure point; a valid start exists iff total gas ≥ total cost.	O(n)	O(1)	reset start when tank goes negative
135	Candy	Assign minimum candies to children in a line: each child gets at least 1 candy, and a child with a higher rating than an adjacent neighbor gets more candies than that neighbor.	Two passes capture the two directions of the constraint — a left-to-right pass enforces "higher than left neighbor gets more", a right-to-left pass enforces "higher than right neighbor gets more". Taking the max of both satisfies both.	O(n)	O(n)	higher than left neighbor
274	H-Index	Given an array of citation counts, find the largest `h` such that at least `h` papers each have at least `h` citations.	After sorting ascending, if a paper at index `i` has `citations[i]` citations, then there are `n − i` papers with at least that many. The h-index is found where `citations[i] >= n − i` first holds.	O(n log n)	O(1)	n-i papers have >= citations[i]
455	Assign Cookies	Each child `i` has a greed factor `g[i]` (minimum cookie size needed); each cookie `j` has size `s[j]`. Each cookie satisfies at most one child. Maximize the number of content children.	Sort both arrays; give the smallest sufficient cookie to the least greedy child. This never wastes a large cookie on a child a small one could satisfy.	O(n log n)	O(1)	Standard
575	Distribute Candies	Distribute `n` candies (the holder eats exactly `n/2`) to maximize the number of distinct candy types eaten.	You can eat at most `n/2` candies, and at best one of each distinct type. So the answer is the smaller of the number of distinct types and `n/2`.	O(n)	O(n)	capped by half the total
605	Can Place Flowers	Given a flowerbed (array of 0s and 1s where 1 is a planted flower), determine if `n` new flowers can be planted without any two flowers being adjacent.	Greedily plant a flower at the first empty plot whose neighbors are also empty — planting as early as possible never blocks a future placement that a later choice would allow.	O(n)	O(1)	plot and both neighbors empty
630	Course Schedule III	Each course has `(duration, lastDay)`. Starting at day 0 and taking courses	Sort by deadline so deadlines are considered in order. Greedily take each course; if total time exceeds the current	O(n log n)	O(n)	over deadline → drop longest

#	Name	Description	Intuition	Time	Space	Key Implementation
		one at a time, find the maximum number of courses that can be taken so each finishes by its `lastDay`.	deadline, drop the previously taken course with the largest duration (a max-heap) — swapping out the longest course frees the most time while keeping the count.			
881	Boats to Save People	Each boat carries at most 2 people with total weight at most `limit`. Given people's weights, find the minimum number of boats to rescue everyone.	Pair the heaviest remaining person with the lightest remaining one if they fit together; otherwise the heaviest goes alone. Sorting plus two pointers makes this pairing optimal.	O(n log n)	O(1)	Standard
986	Interval List Intersections	Given two lists of pairwise-disjoint, sorted intervals, return the intersection of the two lists.	Two pointers walk both sorted lists together. The overlap of the two current intervals is `[max(starts), min(ends)]`, which is valid only when `max(starts) <= min(ends)`. Advance whichever interval ends first.	O(m + n)	O(m + n)	valid overlap
1005	Maximize Sum Of Array After K Negations	Given an array and `k`, perform exactly `k` operations, each negating one element (the same index may be chosen repeatedly), to maximize the array sum.	Flipping the most negative number first yields the biggest gain. After all negatives are flipped (or k runs out), any leftover flips should land on the smallest-magnitude element, since flipping it twice is a no-op.	O(n log n)	O(1)	leftover odd flips hit smallest
1094	Car Pooling	Given trips `[numPassengers, from, to]` and a car capacity, determine whether all trips can be completed without exceeding capacity at any point.	A difference array (sweep line) over locations records passengers boarding at `from` and leaving at `to`. Running the prefix sum gives the occupancy at each point; if it ever exceeds capacity, fail.	O(n + maxLocation)	O(maxLocation)	over capacity → fail
1272	Remove Interval	Given a sorted list of disjoint intervals and an interval `toBeRemoved`, return the set of intervals after removing every point that lies inside `toBeRemoved`.	Each interval is either fully outside the removal range (keep as is) or partially overlaps it (keep the non-overlapping left and/or right slivers). Walk once and emit the surviving pieces.	O(n)	O(n)	keep left sliver
1326	Minimum Number of Taps to Open to Water a Garden	A garden spans `[0, n]`. Tap `i` at position `i` waters `[i - ranges[i], i + ranges[i]]`. Find the minimum number of taps to water the whole garden, or -1 if impossible.	This is a jump-game in disguise: for each starting point record the farthest reach of any tap covering it, then greedily extend coverage layer by layer like minimum jumps.	O(n)	O(n)	gap in coverage → impossible
1353	Maximum Number of Events That Can Be Attended	Each event `[start, end]` can be attended on any single day in `[start, end]`. Attend at most one event per day.	Process days in increasing order; among all events available on the current day, attend the one that ends soonest (a min-heap by end	O(n log n)	O(n)	attend earliest-ending event

#	Name	Description	Intuition	Time	Space	Key Implementation
		Find the maximum number of events you can attend.	day) — saving longer-lived events for later days.			
1488	Avoid Flood in The City	`rains[i] > 0` means lake `rains[i]` fills with rain on day `i`; `rains[i] == 0` is a dry day on which you may dry exactly one lake. A full lake that rains again floods. Return an array where dry days hold the lake to dry (any valid choice) and rain days hold -1, or an empty array if flooding is unavoidable.	When a lake that is already full rains again, you must have used a dry day between its two rains. Greedily assign the earliest available dry day that falls after the lake was last filled — a TreeSet of dry-day indices gives that earliest day.	O(n log n)	O(n)	no dry day available → flood

Two Interval Templates

```
MERGE DIRECTION RULE (for merge-style problems like #56, where both keys work):
    Sort by START  →  traverse FORWARD  (i = 0 → n-1), extend the END of last kept
    Sort by END    →  traverse BACKWARD (i = n-1 → 0), extend the START of last kept
    These two are exact mirror images. See #56 for both written out side by side.
    (NOTE: this mirror only applies to merging. Template 1 below sorts by end but
     still sweeps FORWARD — there the goal is counting, not merging.)

TEMPLATE 1 — Sort by END time, sweep forward, track end of last kept interval
             Use when: maximizing count of non-overlapping intervals
             Greedy insight: earliest-finishing interval leaves most room for future ones

TEMPLATE 2 — Sort by START time, sweep forward, merge when overlapping
             Use when: merging/counting overlapping intervals
             Greedy insight: processing in start order → each interval only needs to
             compare with the last merged result
```

```java
// MENTAL MODEL: sort to expose a local greedy choice, then sweep once making the locally-best pick.
// WHEN: intervals + "max non-overlapping / min arrows / merge / min rooms", or jump/partition sweeps.
// TEMPLATE 1: Sort by end, compare start
Arrays.sort(intervals, (a, b) -> a[1] - b[1]);      // sort by END time
int lastEnd = Integer.MIN_VALUE;
for (int[] interval : intervals) {
    if (interval[0] >= lastEnd) {                    // start >= lastEnd → no overlap → keep
        lastEnd = interval[1];
        // count++ or add to result
    } else {
        // overlap → skip (or count as removed)
    }
}

// TEMPLATE 2: Sort by start, compare end
Arrays.sort(intervals, (a, b) -> a[0] - b[0]);      // sort by START time
List<int[]> result = new ArrayList<>();
for (int[] interval : intervals) {
    if (result.isEmpty() || result.get(result.size()-1)[1] < interval[0]) {
        result.add(interval);                        // no overlap → new interval
    } else {
        result.get(result.size()-1)[1] =             // overlap → extend end
            Math.max(result.get(result.size()-1)[1], interval[1]);
    }
}

// TEMPLATE 3: Sort by start + min-heap of end times (multi-resource)
// USE WHEN counting simultaneous/overlapping resources — "minimum rooms", "max concurrent".
Arrays.sort(intervals, (a, b) -> a[0] - b[0]);
PriorityQueue<Integer> pq = new PriorityQueue<>();   // min-heap of active end times
for (int[] interval : intervals) {
    if (!pq.isEmpty() && pq.peek() <= interval[0])
        pq.poll();                                   // reuse: earliest-ending resource is free
    pq.offer(interval[1]);                           // occupy a resource until interval[1]
}
// pq.size() = resources needed
```

Part 1 — Sort by End Time

#435 Non-overlapping Intervals

Description: Remove the minimum number of intervals to make the rest non-overlapping.
Intuition: Always keep the interval that finishes earliest — it leaves the most room for the rest, so greedily picking by end maximizes how many you keep.
Key: equivalent to keeping the maximum number of non-overlapping intervals. Sort by end, greedily keep each interval whose start ≥ last kept end.

```java
class Solution {
    public int eraseOverlapIntervals(int[][] intervals) {
        Arrays.sort(intervals, (a, b) -> a[1] - b[1]);          // sort by END
        int kept = 0, lastEnd = Integer.MIN_VALUE;
        for (int[] interval : intervals) {
            if (interval[0] >= lastEnd) {                        // no overlap → keep
                lastEnd = interval[1];
                kept++;
            }
            // else: overlap → skip (counted as removed)
        }
        return intervals.length - kept;
    }
}
```

Time O(n log n) | **Space** O(1)

#452 Minimum Number of Arrows to Burst Balloons

Description: Balloons span `[start, end]`. An arrow shot at `x` pops all balloons with `start ≤ x ≤ end`. Minimum arrows to pop all balloons.
Intuition: Shoot the arrow at the earliest end point to pop every balloon overlapping it; only start a new arrow when a balloon begins past the current arrow.
Key: same structure as #435 — sort by end, count groups. One arrow suffices per group. New group starts when `start > current arrow position (= last group's end)`.

```java
class Solution {
    public int findMinArrowShots(int[][] points) {
        Arrays.sort(points, (a, b) -> Integer.compare(a[1], b[1]));  // sort by END (use compare to avo
        int arrows = 1, arrowEnd = points[0][1];
        for (int[] point : points) {
            if (point[0] > arrowEnd) {                           // new group: no overlap with current
                arrows++;
                arrowEnd = point[1];
            }
            // else: same arrow covers this balloon (start <= arrowEnd)
        }
        return arrows;
    }
}
```

Time O(n log n) | **Space** O(1)

#646 Maximum Length of Pair Chain

Description: Given pairs `[a, b]`, form a chain where `b < c` for each consecutive pair `[a,b]` → `[c,d]`. Maximum chain length.
Intuition: Picking the pair that ends earliest each time leaves the most slack for later links, maximizing the chain.
Key: identical to #435 / #452 — sort by end, greedily pick non-overlapping pairs.

```java
class Solution {
    public int findLongestChain(int[][] pairs) {
        Arrays.sort(pairs, (a, b) -> a[1] - b[1]);              // sort by END
        int count = 0, lastEnd = Integer.MIN_VALUE;
        for (int[] pair : pairs) {
            if (pair[0] > lastEnd) {                             // strict: pair[0] > lastEnd
                count++;
                lastEnd = pair[1];
            }
        }
        return count;
    }
}
```

Time O(n log n) | **Space** O(1)

435 vs 452 vs 646 — The One Difference

```
                    #435 Non-overlapping    #452 Min Arrows         #646 Pair Chain
Sort by:            end                     end                     end
Keep condition:     start >= lastEnd        start > arrowEnd        start > lastEnd
Boundary:           >= (inclusive)          > (exclusive)           > (exclusive)
Return:             intervals.length - kept arrows (groups)         count (groups)

Why 435 uses >= but 452/646 use >:
  #435: touching intervals [1,2],[2,3] are non-overlapping (gap = 0 is OK)
  #452: arrow at x=2 pops BOTH [1,2] and [2,3] (touching = same arrow)
  #646: pair chain requires b < c strictly (touching b==c not allowed)
```

Part 2 — Sort by Start Time

#56 Merge Intervals

Description: Merge all overlapping intervals. Return the minimum set of non-overlapping intervals.
Intuition: sort to make overlaps adjacent, then sweep once — each interval only ever compares against the most recently kept one.

> **The direction rule — sort key dictates traversal direction:**
>
> - **Sort by START → traverse forward** (`i = 0 → n-1`). The newest interval can only extend the last kept interval's **END**, so you compare `interval.start` against `last.end` and update `last[1]`.
> - **Sort by END → traverse backward** (`i = n-1 → 0`). The newest interval can only extend the last kept interval's **START**, so you compare `interval.end` against `last.start` and update `last[0]`.
>
> Both are symmetric and equally valid. Pick the one whose sort key you already have.

Method A — Sort by START, traverse forward

Compare current `start` to last kept `end`; on overlap, extend the **end**.

```java
class Solution {
    public int[][] merge(int[][] intervals) {
        Arrays.sort(intervals, (a, b) -> a[0] - b[0]);          // sort by START
        List<int[]> result = new ArrayList<>();
        for (int[] interval : intervals) {                       // ← traverse FORWARD
            if (result.isEmpty()) {
                result.add(interval);
                continue;
            }
            int[] last = result.get(result.size() - 1);
            if (last[1] < interval[0]) {                         // no overlap (last.end < current.star
                result.add(interval);
            } else {
                last[1] = Math.max(last[1], interval[1]);        // overlap → extend END
            }
        }
        return result.toArray(new int[0][]);
    }
}
```

Method B — Sort by END, traverse backward

Mirror image: compare current `end` to last kept `start`; on overlap, extend the **start**.

```java
class Solution {
    public int[][] merge(int[][] intervals) {
        Arrays.sort(intervals, (a, b) -> a[1] - b[1]);          // sort by END
        List<int[]> result = new ArrayList<>();
        for (int i = intervals.length - 1; i >= 0; i--) {        // ← traverse BACKWARD
            if (result.isEmpty()) {
                result.add(intervals[i]);
                continue;
            }
            int[] last = result.get(result.size() - 1);
            int[] current = intervals[i];
            if (last[0] > current[1]) {                          // no overlap (last.start > current.en
                result.add(current);
            } else {
                last[0] = Math.min(last[0], current[0]);         // overlap → extend START
            }
        }
        return result.toArray(new int[0][]);
    }
}
```

Time O(n log n) | **Space** O(n)

#57 Insert Interval

Description: Insert a new interval into a sorted list of non-overlapping intervals; merge if necessary. The input is already sorted — no sort needed.
Intuition: Walk the sorted list in three phases — copy everything that ends before the new interval, absorb everything that overlaps it, then copy the rest.
Key: three phases: (1) add all intervals that end before new starts, (2) merge all overlapping, (3) add remaining.

```java
class Solution {
    public int[][] insert(int[][] intervals, int[] newInterval) {
        List<int[]> list = new ArrayList<>();
        for(int[] interval : intervals) {
            if(interval[1] < newInterval[0]) {
                list.add(interval);
            } else if (interval[0] > newInterval[1]) {
                list.add(newInterval);
                newInterval = interval;
            } else {
                newInterval[0] = Math.min(newInterval[0], interval[0]);
                newInterval[1] = Math.max(newInterval[1], interval[1]);
            }
        }
        list.add(newInterval);
        return list.toArray(new int[list.size()][2]);
    }
}
```

Time O(n) | **Space** O(n)

Part 3 — Sort by Start + Min-Heap (Multi-Resource)

#253 Meeting Rooms II

Description: Minimum number of meeting rooms required to host all meetings.
Intuition: USE WHEN counting simultaneous/overlapping resources — "minimum rooms", "max concurrent". The heap size at any moment is exactly how many meetings are running at once.
Template: sort by start. Min-heap tracks end times of active meetings. When a new meeting starts after the earliest-ending meeting finishes, reuse that room.

```java
class Solution {
    public int minMeetingRooms(int[][] intervals) {
        Arrays.sort(intervals, (a, b) -> a[0] - b[0]);          // sort by START
        PriorityQueue<Integer> pq = new PriorityQueue<>();        // min-heap of end times
        for (int[] interval : intervals) {
            if (!pq.isEmpty() && pq.peek() <= interval[0])
                pq.poll();                                       // ← reuse: earliest room is free
            pq.offer(interval[1]);                               // ← occupy room until this end
        }
        return pq.size();                                        // ← rooms in use = heap size
    }
}
```

Time O(n log n) | **Space** O(n)

Sort-by-End vs Sort-by-Start+Heap

	Sort by End	Sort by Start + Heap
Goal:	count non-overlapping	count simultaneous active
Tracks:	lastEnd (scalar)	all active end times (heap)
Question answered:	max intervals I can keep	min rooms (resources) needed
Examples:	#435, #452, #646	#253 Meeting Rooms

Part 4 — Non-interval Greedy

#55 Jump Game

Description: Each element is the max jump from that position. Can you reach the last index?
Intuition: Sweep left to right tracking the farthest index reachable; if you ever stand past that frontier you're stuck.
Key: track the farthest index reachable so far. If current index exceeds it, you're stuck.

```
class Solution {
    public boolean canJump(int[] nums) {
        int maxReach = 0;
        for (int i = 0; i < nums.length; i++) {
            if (i > maxReach) return false;                    // ← can't reach index i
            maxReach = Math.max(maxReach, i + nums[i]);
        }
        return true;
    }
}
```

Time O(n) | **Space** O(1)

#45 Jump Game II

Description: Minimum jumps to reach the last index.
Intuition: Treat each jump as a BFS layer — extend through the whole current reach before incrementing the jump count.
Key: BFS layer-by-layer. `currentEnd` = farthest we can reach with current jumps. `farthest` = farthest reachable with one more jump from anywhere in current layer. When we reach `currentEnd`, we must jump.

```
class Solution {
    public int jump(int[] nums) {
        int jumps = 0, currentEnd = 0, farthest = 0;
        for (int i = 0; i < nums.length - 1; i++) {
            farthest = Math.max(farthest, i + nums[i]);
            if (i == currentEnd) {                             // ← exhausted current layer
                jumps++;
                currentEnd = farthest;                         // ← extend to next layer
            }
        }
        return jumps;
    }
}
```

Time O(n) | **Space** O(1)

#763 Partition Labels

Description: Partition string into as many parts as possible so each letter appears in at most one part. Return the size of each part.
Intuition: A partition can't close until you pass the last occurrence of every letter seen inside it, so extend `end` to that frontier and cut when you reach it.
Key: precompute the last occurrence of each character. Extend the current partition's end to `max(end, last[c])` for each character. Close partition when `i == end`.

```java
class Solution {
    public List<Integer> partitionLabels(String s) {
        int[] last = new int[26];
        for (int i = 0; i < s.length(); i++) last[s.charAt(i) - 'a'] = i;  // ← last occurrence
        List<Integer> result = new ArrayList<>();
        int start = 0, end = 0;
        for (int i = 0; i < s.length(); i++) {
            end = Math.max(end, last[s.charAt(i) - 'a']);       // ← must include last of this char
            if (i == end) {                                      // ← reached end of partition
                result.add(end - start + 1);
                start = i + 1;
            }
        }
        return result;
    }
}
```

Time O(n) | **Space** O(1)

#406 Queue Reconstruction by Height

Description: People given as `[h, k]` (height h, k taller/equal people in front). Reconstruct the queue.
Intuition: Place tallest first so that inserting a person at index `k` is always correct — only taller/equal people are already placed, and shorter insertions later don't disturb their counts.
Key: sort descending by height (ties: ascending by k). Insert each person at index `k`. Since taller people are placed first, inserting at position `k` is always valid — there are already exactly k people ≥ this height already placed.

```java
class Solution {
    public int[][] reconstructQueue(int[][] people) {
        Arrays.sort(people, (a, b) -> a[0] != b[0] ? b[0] - a[0] : a[1] - b[1]);  // ← tall first, ties
        List<int[]> result = new ArrayList<>();
        for (int[] p : people)
            result.add(p[1], p);                                 // ← insert at index k
        return result.toArray(new int[0][]);
    }
}
```

Time O(n^2) | **Space** O(n)

Greedy Pattern Identifier

Signal	Template
"minimum removal to make non-overlapping"	Sort by end, keep non-overlapping (Template 1)
"minimum resources (arrows, groups)"	Sort by end, count groups (Template 1)
"merge overlapping intervals"	Sort by start, extend or add (Template 2)
"minimum rooms/machines"	Sort by start + min-heap (Template 3)
"can you reach the end?"	Track max reachable index
"minimum steps to reach the end?"	BFS layers via `currentEnd`
"partition string by last occurrence"	Track `last[]` array + extend `end`
"reconstruct order by two constraints"	Sort by one constraint, insert by the other
"can you complete the circuit / circular route?"	Track cumulative tank; reset start when tank < 0
"can person attend all meetings (no overlap)?"	Sort by start, compare adjacent intervals

Part 5 — Overlap & Circular Patterns

#252 Meeting Rooms

Description: Given an array of meeting intervals, determine if a person could attend all meetings (no two meetings overlap).

Intuition: After sorting by start, any overlap must be between adjacent intervals, so one linear scan comparing neighbors is enough.

Algorithm: Sort by start time. If any meeting starts before the previous one ends, there is an overlap — return false.

```
class Solution {
    public boolean canAttendMeetings(int[][] intervals) {
        Arrays.sort(intervals, (a, b) -> a[0] - b[0]);    // ← sort by start time
        for (int i = 1; i < intervals.length; i++)
            if (intervals[i][0] < intervals[i-1][1]) return false;  // ← VARIATION: start < previous en
        return true;
    }
}
```

Time O(n log n) | **Space** O(1)

#134 Gas Station

Description: There are n gas stations on a circular route. `gas[i]` is the gas at station i; `cost[i]` is the gas needed to travel from i to i+1. Return the starting station index if you can complete the circuit, or -1 if not.

Intuition: If the tank ever drops below 0, no station between the old start and here can work, so jump the start past the failure point; a valid start exists iff total gas ≥ total cost.

Algorithm: Greedy. Track cumulative tank. When tank goes negative, reset start to `i+1` and reset tank. If total gas >= total cost, a valid start exists (the last reset start).

```
class Solution {
    public int canCompleteCircuit(int[] gas, int[] cost) {
        int total = 0, tank = 0, start = 0;
        for (int i = 0; i < gas.length; i++) {
            int diff = gas[i] - cost[i];
            total += diff;
            tank  += diff;
            if (tank < 0) {
                start = i + 1;  // ← VARIATION: reset start when tank goes negative
                tank  = 0;
            }
        }
        return total >= 0 ? start : -1;  // ← VARIATION: if total >= 0, solution exists
    }
}
```

Time O(n) | **Space** O(1)

Additional Reference Problems

#135 Candy

Description: Assign minimum candies to children in a line: each child gets at least 1 candy, and a child with a higher rating than an adjacent neighbor gets more candies than that neighbor.

Intuition: Two passes capture the two directions of the constraint — a left-to-right pass enforces "higher than left neighbor gets more", a right-to-left pass enforces "higher than right neighbor gets more". Taking the max of both satisfies both.

Algorithm: Initialize all candies to 1. Sweep left→right giving `candies[i] = candies[i−1] + 1` when `ratings[i] > ratings[i−1]`. Sweep right→left taking `max(candies[i], candies[i+1] + 1)` when `ratings[i] > ratings[i+1]`. Sum.

```
class Solution {
    public int candy(int[] ratings) {
        int n = ratings.length;
        int[] candies = new int[n];
        Arrays.fill(candies, 1);
        for (int i = 1; i < n; i++) {
            if (ratings[i] > ratings[i - 1]) {                  // ← VARIATION: higher than left neighb
                candies[i] = candies[i - 1] + 1;
            }
        }
        for (int i = n - 2; i >= 0; i--) {
            if (ratings[i] > ratings[i + 1]) {                  // ← VARIATION: higher than right neigh
                candies[i] = Math.max(candies[i], candies[i + 1] + 1);
            }
        }
        int result = 0;
        for (int i = 0; i < n; i++) {
            result += candies[i];
        }
        return result;
    }
}
```

Time O(n) | **Space** O(n)

#274 H-Index

Description: Given an array of citation counts, find the largest `h` such that at least `h` papers each have at least `h` citations.

Intuition: After sorting ascending, if a paper at index `i` has `citations[i]` citations, then there are `n − i` papers with at least that many. The h-index is found where `citations[i] >= n − i` first holds.

Algorithm: Sort ascending. Scan; the first index `i` where `citations[i] >= n − i` gives `h = n − i`, the largest valid h. If none, h is 0.

```java
class Solution {
    public int hIndex(int[] citations) {
        Arrays.sort(citations);                                 // ← sort ascending
        int n = citations.length;
        for (int i = 0; i < n; i++) {
            if (citations[i] >= n - i) {                        // ← VARIATION: n-i papers have >= cita
                return n - i;
            }
        }
        return 0;
    }
}
```

Time O(n log n) | **Space** O(1)

#455 Assign Cookies

Description: Each child `i` has a greed factor `g[i]` (minimum cookie size needed); each cookie `j` has size `s[j]`. Each cookie satisfies at most one child. Maximize the number of content children.

Intuition: Sort both arrays; give the smallest sufficient cookie to the least greedy child. This never wastes a large cookie on a child a small one could satisfy.

Algorithm: Sort `g` and `s`. Two pointers: advance the cookie pointer always; advance the child pointer (counting a satisfied child) only when the current cookie satisfies the current child.

```java
class Solution {
    public int findContentChildren(int[] g, int[] s) {
        Arrays.sort(g);                                         // ← sort children by greed
        Arrays.sort(s);                                         // ← sort cookies by size
        int i = 0, j = 0, result = 0;
        while (i < g.length && j < s.length) {
            if (s[j] >= g[i]) {                                 // ← cookie satisfies child
                result++;
                i++;
            }
            j++;
        }
        return result;
    }
}
```

Time O(n log n) | **Space** O(1)

#575 Distribute Candies

Description: Distribute `n` candies (the holder eats exactly `n/2`) to maximize the number of distinct candy types eaten.

Intuition: You can eat at most `n/2` candies, and at best one of each distinct type. So the answer is the smaller of the number of distinct types and `n/2`.

Algorithm: Count distinct types with a set. Return `min(distinct types, n/2)`.

```java
class Solution {
    public int distributeCandies(int[] candyType) {
        Set<Integer> types = new HashSet<>();
        for (int i = 0; i < candyType.length; i++) {
            types.add(candyType[i]);
        }
        return Math.min(types.size(), candyType.length / 2);    // ← VARIATION: capped by half the tota
    }
}
```

Time O(n) | **Space** O(n)

#605 Can Place Flowers

Description: Given a flowerbed (array of 0s and 1s where 1 is a planted flower), determine if `n` new flowers can be planted without any two flowers being adjacent.

Intuition: Greedily plant a flower at the first empty plot whose neighbors are also empty — planting as early as possible never blocks a future placement that a later choice would allow.

Algorithm: Scan; at each empty plot whose left and right (treating out-of-bounds as empty) are empty, plant it (set to 1) and decrement `n`. Succeed early if `n <= 0`.

```java
class Solution {
    public boolean canPlaceFlowers(int[] flowerbed, int n) {
        for (int i = 0; i < flowerbed.length; i++) {
            boolean leftEmpty = (i == 0) || (flowerbed[i - 1] == 0);
            boolean rightEmpty = (i == flowerbed.length - 1) || (flowerbed[i + 1] == 0);
            if (flowerbed[i] == 0 && leftEmpty && rightEmpty) { // ← VARIATION: plot and both neighbors
                flowerbed[i] = 1;                               // ← plant greedily
                n--;
            }
            if (n <= 0) {
                return true;
            }
        }
        return n <= 0;
    }
}
```

Time O(n) | **Space** O(1)

#630 Course Schedule III

Description: Each course has `(duration, lastDay)`. Starting at day 0 and taking courses one at a time, find the maximum number of courses that can be taken so each finishes by its `lastDay`.

Intuition: Sort by deadline so deadlines are considered in order. Greedily take each course; if total time exceeds the current deadline, drop the previously taken course with the largest duration (a max-heap) — swapping out the longest course frees the most time while keeping the count.

Algorithm: Sort by `lastDay`. Maintain a max-heap of taken durations and running `time`. Add each course; if `time > lastDay`, pop the longest course and subtract it. Heap size is the answer.

```java
class Solution {
    public int scheduleCourse(int[][] courses) {
        Arrays.sort(courses, (a, b) -> a[1] - b[1]);            // ← sort by deadline
        PriorityQueue<Integer> maxHeap = new PriorityQueue<>((a, b) -> b - a);
        int time = 0;
        for (int[] course : courses) {
            time += course[0];
            maxHeap.offer(course[0]);
            if (time > course[1]) {                             // ← VARIATION: over deadline → drop lo
                time -= maxHeap.poll();
            }
        }
        return maxHeap.size();
    }
}
```

Time O(n log n) | **Space** O(n)

#881 Boats to Save People

Description: Each boat carries at most 2 people with total weight at most `limit`. Given people's weights, find the minimum number of boats to rescue everyone.

Intuition: Pair the heaviest remaining person with the lightest remaining one if they fit together; otherwise the heaviest goes alone. Sorting plus two pointers makes this pairing optimal.

Algorithm: Sort ascending. Two pointers from both ends; if `people[i] + people[j] <= limit` the lightest also boards (advance `i`). Always send the heaviest (decrement `j`) and count a boat.

```java
class Solution {
    public int numRescueBoats(int[] people, int limit) {
        Arrays.sort(people);                                    // ← sort ascending
        int i = 0, j = people.length - 1, result = 0;
        while (i <= j) {
            if (people[i] + people[j] <= limit) {               // ← lightest fits with heaviest
                i++;
            }
            j--;                                                // ← heaviest always boards
            result++;
        }
        return result;
    }
}
```

Time O(n log n) | **Space** O(1)

#986 Interval List Intersections

Description: Given two lists of pairwise-disjoint, sorted intervals, return the intersection of the two lists.

Intuition: Two pointers walk both sorted lists together. The overlap of the two current intervals is `[max(starts), min(ends)]`, which is valid only when `max(starts) <= min(ends)`. Advance whichever interval ends first.

Algorithm: Two pointers `i`, `j`. Compute `lo = max(starts)`, `hi = min(ends)`; if `lo <= hi`, record `[lo, hi]`. Advance the pointer of the interval with the smaller end.

```java
class Solution {
    public int[][] intervalIntersection(int[][] firstList, int[][] secondList) {
        List<int[]> result = new ArrayList<>();
        int i = 0, j = 0;
        while (i < firstList.length && j < secondList.length) {
            int lo = Math.max(firstList[i][0], secondList[j][0]);  // ← latest start
            int hi = Math.min(firstList[i][1], secondList[j][1]);  // ← earliest end
            if (lo <= hi) {                                        // ← VARIATION: valid overlap
                result.add(new int[]{lo, hi});
            }
            if (firstList[i][1] < secondList[j][1]) {              // ← advance interval that ends first
                i++;
            } else {
                j++;
            }
        }
        return result.toArray(new int[0][]);
    }
}
```

Time O(m + n) | **Space** O(m + n)

#1005 Maximize Sum Of Array After K Negations

Description: Given an array and `k`, perform exactly `k` operations, each negating one element (the same index may be chosen repeatedly), to maximize the array sum.

Intuition: Flipping the most negative number first yields the biggest gain. After all negatives are flipped (or k runs out), any leftover flips should land on the smallest-magnitude element, since flipping it twice is a no-op.

Algorithm: Sort ascending. Flip negatives left to right while `k > 0`. If `k` is still positive and odd, negate the now-smallest absolute value element once. Sum.

```java
class Solution {
    public int largestSumAfterKNegations(int[] nums, int k) {
        Arrays.sort(nums);                                         // ← sort ascending
        for (int i = 0; i < nums.length && k > 0; i++) {
            if (nums[i] < 0) {                                     // ← flip most-negative first
                nums[i] = -nums[i];
                k--;
            }
        }
        int sum = 0, min = Integer.MAX_VALUE;
        for (int i = 0; i < nums.length; i++) {
            sum += nums[i];
            min = Math.min(min, nums[i]);
        }
        if (k % 2 == 1) {                                          // ← VARIATION: leftover odd flips hit
            sum -= 2 * min;
        }
        return sum;
    }
}
```

Time O(n log n) | **Space** O(1)

#1094 Car Pooling

Description: Given trips `[numPassengers, from, to]` and a car capacity, determine whether all trips can be completed without exceeding capacity at any point.

Intuition: A difference array (sweep line) over locations records passengers boarding at `from` and leaving at `to`. Running the prefix sum gives the occupancy at each point; if it ever exceeds capacity, fail.

Algorithm: Add `num` at `from` and subtract `num` at `to` in a difference array. Prefix-sum across locations; return false if any running total exceeds capacity.

```
class Solution {
    public boolean carPooling(int[][] trips, int capacity) {
        int[] diff = new int[1001];
        for (int[] trip : trips) {
            diff[trip[1]] += trip[0];                          // ← board at from
            diff[trip[2]] -= trip[0];                          // ← leave at to
        }
        int passengers = 0;
        for (int i = 0; i < diff.length; i++) {
            passengers += diff[i];                             // ← running occupancy
            if (passengers > capacity) {                       // ← VARIATION: over capacity → fail
                return false;
            }
        }
        return true;
    }
}
```

Time O(n + maxLocation) | **Space** O(maxLocation)

#1272 Remove Interval

Description: Given a sorted list of disjoint intervals and an interval `toBeRemoved`, return the set of intervals after removing every point that lies inside `toBeRemoved`.

Intuition: Each interval is either fully outside the removal range (keep as is) or partially overlaps it (keep the non-overlapping left and/or right slivers). Walk once and emit the surviving pieces.

Algorithm: For each interval, if it is entirely before or after `toBeRemoved`, keep it. Otherwise keep the left piece `[start, toBeRemoved[0]]` if it has positive length and the right piece `[toBeRemoved[1], end]` if it has positive length.

```
class Solution {
    public List<List<Integer>> removeInterval(int[][] intervals, int[] toBeRemoved) {
        List<List<Integer>> result = new ArrayList<>();
        for (int[] interval : intervals) {
            if (interval[1] <= toBeRemoved[0] || interval[0] >= toBeRemoved[1]) {
                result.add(Arrays.asList(interval[0], interval[1]));  // ← no overlap → keep whole
            } else {
                if (interval[0] < toBeRemoved[0]) {            // ← VARIATION: keep left sliver
                    result.add(Arrays.asList(interval[0], toBeRemoved[0]));
                }
                if (interval[1] > toBeRemoved[1]) {            // ← VARIATION: keep right sliver
                    result.add(Arrays.asList(toBeRemoved[1], interval[1]));
                }
            }
        }
        return result;
    }
}
```

Time O(n) | **Space** O(n)

#1326 Minimum Number of Taps to Open to Water a Garden

Description: A garden spans `[0, n]`. Tap `i` at position `i` waters `[i - ranges[i], i + ranges[i]]`. Find the minimum number of taps to water the whole garden, or -1 if impossible.

Intuition: This is a jump-game in disguise: for each starting point record the farthest reach of any tap covering it, then greedily extend coverage layer by layer like minimum jumps.

Algorithm: Build `maxReach[i]` = farthest right coverage of any tap whose left edge is at or before `i`. Sweep like Jump Game II: track `currentEnd` and `farthest`; when reaching `currentEnd`, increment taps and jump. Fail if `farthest` stalls before `n`.

```
class Solution {
    public int minTaps(int n, int[] ranges) {
        int[] maxReach = new int[n + 1];
        for (int i = 0; i <= n; i++) {
            int left = Math.max(0, i - ranges[i]);
            int right = Math.min(n, i + ranges[i]);
            maxReach[left] = Math.max(maxReach[left], right);   // ← farthest reach from this left edge
        }
        int taps = 0, currentEnd = 0, farthest = 0;
        for (int i = 0; i <= n; i++) {
            if (i > farthest) {                                 // ← VARIATION: gap in coverage → impos
                return -1;
            }
            farthest = Math.max(farthest, maxReach[i]);
            if (i == currentEnd && i < n) {                     // ← exhausted current layer → open a t
                taps++;
                currentEnd = farthest;
            }
        }
        return currentEnd >= n ? taps : -1;
    }
}
```

Time O(n) | **Space** O(n)

#1353 Maximum Number of Events That Can Be Attended

Description: Each event `[start, end]` can be attended on any single day in `[start, end]`. Attend at most one event per day. Find the maximum number of events you can attend.

Intuition: Process days in increasing order; among all events available on the current day, attend the one that ends soonest (a min-heap by end day) — saving longer-lived events for later days.

Algorithm: Sort events by start. For each day, push all events that have started into a min-heap keyed on end day, discard events whose end day has passed, then attend (poll) one event and count it.

```java
class Solution {
    public int maxEvents(int[][] events) {
        Arrays.sort(events, (a, b) -> a[0] - b[0]);             // ← sort by start day
        PriorityQueue<Integer> minHeap = new PriorityQueue<>();  // ← min-heap of end days
        int i = 0, day = 0, result = 0, n = events.length;
        while (i < n || !minHeap.isEmpty()) {
            if (minHeap.isEmpty()) {
                day = events[i][0];                             // ← jump to next event's start
            }
            while (i < n && events[i][0] <= day) {              // ← add all events started by today
                minHeap.offer(events[i][1]);
                i++;
            }
            minHeap.poll();                                     // ← VARIATION: attend earliest-ending
            result++;
            day++;
            while (!minHeap.isEmpty() && minHeap.peek() < day) { // ← drop events that already expired
                minHeap.poll();
            }
        }
        return result;
    }
}
```

Time O(n log n) | **Space** O(n)

#1488 Avoid Flood in The City

Description: `rains[i] > 0` means lake `rains[i]` fills with rain on day `i`; `rains[i] == 0` is a dry day on which you may dry exactly one lake. A full lake that rains again floods. Return an array where dry days hold the lake to dry (any valid choice) and rain days hold -1, or an empty array if flooding is unavoidable.

Intuition: When a lake that is already full rains again, you must have used a dry day between its two rains. Greedily assign the earliest available dry day that falls after the lake was last filled — a TreeSet of dry-day indices gives that earliest day.

Algorithm: Track each full lake's last-rain day in a map and dry-day indices in a TreeSet. On a rain day for an already-full lake, find the smallest dry day after its last rain; if none exists, flooding is unavoidable. Use that dry day for this lake. Leftover dry days dry any lake (default 1).

```java
class Solution {
    public int[] avoidFlood(int[] rains) {
        int n = rains.length;
        int[] result = new int[n];
        Map<Integer, Integer> fullDay = new HashMap<>();        // ← lake → day it was last filled
        TreeSet<Integer> dryDays = new TreeSet<>();             // ← indices of available dry days
        for (int i = 0; i < n; i++) {
            if (rains[i] == 0) {
                dryDays.add(i);
                result[i] = 1;                                  // ← default; may be overwritten
            } else {
                result[i] = -1;
                int lake = rains[i];
                if (fullDay.containsKey(lake)) {                // ← lake already full → must dry it ea
                    Integer dry = dryDays.higher(fullDay.get(lake));
                    if (dry == null) {                          // ← VARIATION: no dry day available →
                        return new int[0];
                    }
                    result[dry] = lake;                         // ← use that dry day for this lake
                    dryDays.remove(dry);
                }
                fullDay.put(lake, i);
            }
        }
        return result;
    }
}
```

Time O(n log n) | **Space** O(n)